



Homework 1 Report

Algebraic Algorithms For Cryptology

Ahmad Shami

22nd March 2026

Exercise 1: Implementation of $\mathbb{Z}/n\mathbb{Z}$ and RSA.....	3
(a) Expectations from ring element n.....	3
(b) Implementation of Base Functions.....	3
(c) Modular Exponentiation.....	5
(d) Test For $\mathbb{Z}/n\mathbb{Z}$	6
(e) RSA Implementation.....	6
(f) RSA Demo.....	6
(g) Implementation of Multi-precision Integers.....	7
(h) Multi-precision Integers Tests.....	8
Ex 2: Implementation of binary Galois fields.....	10
(a) Definitions of the Field F_{128}	10
(b) Choosing the Constructor Polynomial of the Field.....	10
(c) Implementation of the Field.....	10
(d) f_{128} Demo.....	12
(e) Zech-Log representation.....	12
(f) Condition of Polynomials in this Field.....	12
(g) Zech-Log Implementation.....	13
(h) Zech-Log Demo.....	14
Appendix.....	15
Links and Resources.....	15
Compiling the project with multi-precision.....	15
Project structure.....	16

Exercise 1: Implementation of $\mathbb{Z}/n\mathbb{Z}$ and RSA

(a) Expectations from ring element n

The main implementation rounds about `uint64_t`, the values of n could theoretically be between $1 \dots 2^{64} - 1$. But some operations like multiplications could result in values higher than 2^{64} which can easily overflow.

Suppose we have this operation:

$$X = 2^{32} + 1234$$

$$Y = 2^{32} + 100$$

Doing a multiplication on these numbers $X * Y$ will result in an integer that is with higher value than 2^{64} . To constrain these values, n should force them to be in a ring with boundary less than 2^{32} (i.e $n < 2^{32}$).

(b) Implementation of Base Functions

The implementation of the functions first started with simple naive approaches. For example, the addition just had the following shape:

```
Element add(Element x, Element y, Element n) {  
    return (x + y) % n;  
}
```

But this approach fell apart when large values of x and y were used for example when $x + y > 2^{64}$.

The modulo operation can propagate into additions:

$$(X + Y) \bmod N = (X \bmod N) + (Y \bmod N)$$

That could be done via the modulo operator being able to propagate into additions. The idea is to reduce X and Y until they both are less than 2^{32} as so:

```

Element add(Element x, Element y, Element n) {
    // reduce x and y to be sure that they will not overflow

    // x + y > UINT64_MAX -> x > UINT64_MAX - y -> reduce
    while(x > (UINT64_MAX) - y || (UINT64_MAX) - y < UINT64_MAX/2){
        x = ((x >> 1) % n) + ((x >> 1) % n) + (x & 1);
        y = ((y >> 1) % n) + ((y >> 1) % n) + (y & 1);
    }

    return (x + y) % n;
}

```

Essentially we split X and Y in half and modulo the halves (also preserving the LSB as we are dividing by 2 for odd numbers).

$$X \bmod N = X/2 \bmod N + X/2 \bmod N$$

$$Y \bmod N = Y/2 \bmod N + Y/2 \bmod N$$

And we attempted to use that for multiplication as well:

```

Element mul(Element x, Element y, Element n){

    while(x > (UINT64_MAX) / y || y > (UINT64_MAX) / x){
        x = ((x >> 1) % n) + ((x >> 1) % n) + (x & 1);
        y = ((y >> 1) % n) + ((y >> 1) % n) + (y & 1);
    }
    return (x * y) % n;
}

```

But in tests (specifically when used in exponentiation) it proved to be very slow. So ultimately we settled down on the implementation discussed in class for fields and adapted it to the ring:

```

Element mul(Element x, Element y, Element n) {
    Element c = 0; // accumulator
    x = x % n; // reduce x if it is larger than n (just having x as is could
    overflow since we add x to it later)

    while (y > 0) { // we loop over the bit size of y from msb
        if (y & 1)

```

```

        c = (c + x) % n; // mulByX here is just adding another X, we mod
to reduce the result below n to avoid overflows again

        x = (x + x) % n; // mulByX again with mod
        y >>= 1; // reduce y
    }

    return c;
}

```

This implementation just uses arithmetic operations instead of the bitwise operations discussed in class. Also we tried to implement Karatsuba's multiplication algorithm based on an implementation we found online[1] that I related to the equation in TD1.

For the inversion function, we implemented a GCD recursive extended euclidean division function adapted from this source[2]. This function allows the extraction of the Bezout coefficients that will be used in the invert function. *isInvertible* function could make use of the result of the GCD function (if the result is 1 -> the arguments i.e the element and n are coprime -> invertible).

The invert function simply makes use of the u Bezout coefficient as it represents the inverse of an element in a ring.

(c) Modular Exponentiation

For this part we implemented two functions. The first is a naive implementation of an iterative multiplication (proved to be so slow with larger integers like the ones used later for RSA), and another that is identical to the multiplication function, except it does a multiplication on x over each iteration rather than an addition:

```

Element modexp(Element x, uint64_t k, Element n){
    Element res = x;
    if(k <= 0)
        return 1;
    for(uint64_t i = 0; i < k - 1; i++)
        res = mul_karatsuba(res, x, n);
    return res;
}

```

```

}

Element fast_modexp(Element x, uint64_t k, Element n){
    Element result = 1;
    x = x % n;

    while(k > 0){
        if(k & 1)
            result = mul(result, x, n);
        x = mul(x,x,n);
        k = k >> 1;
    }

    return result;
}

```

(d) Test For $\mathbb{Z}/n\mathbb{Z}$

The test file created tests the functionality of the functions using $n < 2^{32}$ and with values high enough that they would cause overflows in case n was not chosen carefully. Also it has a section related to the multi-precision implementation I attempted (more on that later).

(e) RSA Implementation

In this section, the implementation of the functions were as described in the lecture slides. The only thing that we had to look up was the value of e , and according to this [post\[3\]](#), the value 65537 is a generally good choice as it is prime and will have low calculation overhead due to its relatively small size.

(f) RSA Demo

For the demo, it is as simple as using the functions. First we encrypt, then decrypt the ciphertext, then compare the result with the original message. The test proved successful on a couple of prime number combinations:

```
./rsa-demo
Using primes:
p = 804582047
q = 981009509

phi = 789302637092093368
e*d mod phi = 1
Generated RSA parameters:
n = 789302638877684923
e = 65537
d = 271944909677578593

Original message: 123456
Encrypted message: 440390669515595382
Decrypted message: 123456
Success: decrypted message matches original.
```

(g) Implementation of Multi-precision Integers

In this part we opted to use an implementation we found online[4], where most of the basic operations (Boolean, bitwise and arithmetic) are implemented, and we just extended them a little bit to do multiplication, GCD, Extended GCD and modular exponentiation.

Note that this is in no way purely my code. I simply used it to explore how it would behave in tests and if I will manage to integrate it into the RSA tests. Please ignore this section if you feel like this falls under plagiarism (Although the author allows any use of the code in the License Agreement[5]). All credits are attributed to the original author[6].

The essence of this implementation is to represent each integer as an array of machine native integer sizes. Then each operation on these integers will work on each array independently unless there are carries or borrows.

In this implementation, we opted to re-implement the multiplication function as the bitwise implementation used in the 64-bit section for speed.

The GCD function is just the same as the previous 64-bit function just in an iterative fashion (as we found it easier to track that way in the multi-precision world).

For the Extended GCD function, we had to extend the implementation to also have a structure that tracks the sign of numbers (used for Bezout coefficients as they could be negative).

And finally we implemented a fast modular exponentiation function similar to the one already implemented just applying the multi-precision integer functions and operations.

(h) Multi-precision Integers Tests

We extended the tests for $\mathbb{Z}/n\mathbb{Z}$ and RSA to make use of the functions and test whether they behave as expected. A sanity check we added in *test-znz* is to see if $\gcd(x,y) = u*x + v*y$ after doing extended euclidean division.

```
./test-znz
Working in  $\mathbb{Z}/17\mathbb{Z}$ 

add(5,9223372036854775818) mod 17 = 7
mul(5,9223372036854775818) mod 17 = 10
mul_karatsuba(5,9223372036854775818) mod 17 = 12
neg(5) mod 17 = 12
gcd(5,17) = 1
Bezout coefficients:  $5*7 + 17*-2 = 1$ 
5 is invertible mod 17
inverse(5) mod 17 = 7
 $5^5 \bmod 17 = 14$ 
 $5^5 \bmod 17$  (fast mod exp) = 14
Multiprecision tests:
mulpr_add(5,9223372036854775818) = 15
mulpr_sub(5,9223372036854775818) = -5
mulpr_mul(5,9223372036854775818) = 50
mulpr_div(5,9223372036854775818) = 0
mulpr_mod(5,9223372036854775818) = 5
mulpr_gcd(5,9223372036854775818) = 1
mulpr_exgcd(5,17):
gcd = 1
u = -858993463
v = 2
gcd(x,y) and  $ux + vy$  are equal!
mulpr_modexp(5,5) mod 17 = 14
```

Unfortunately we didn't have time to implement good printing functions that show the actual values, rather we opted to convert to signed integers which truncated values making the displayed values mostly incorrect.

Also we tried to integrate the multi-precision implementation to the RSA tests but unfortunately it didn't work for this implementation and we didn't have time to debug:

```
./rsa-demo
Using primes:
p = 804582047
q = 981009509

phi = 789302637092093368
e*d mod phi = 1
Generated RSA parameters:
n = 789302638877684923
e = 65537
d = 271944909677578593

Original message: 123456
Encrypted message: 440390669515595382
Decrypted message: 123456
Success: decrypted message matches original.

Attempting rsa with multiprecision integers implementation:
Using primes:
p = 313486403
q = 521502889

Generated RSA parameters:
n = 175044667
e = 65537
d = 855414623

Original message: 123456
Encrypted message: 3460100619
Decrypted message: 3681690683
Error: decryption failed.
```

Please refer to the appendix to learn more about compilation instructions to enable the multi-precision extension.

Ex 2: Implementation of binary Galois fields

(a) Definitions of the Field F_{128}

- 1) The elements are represented over a field F_{2^7} and can be constructed going up from the field F_2 .
- 2) Addition in this field is a simple bitwise XOR of the two operands
- 3) multiplication can be done by a series of bit shifts to the right with each multiplication by a polynomial X
- 4) inversion can be obtained via the extended euclidean division with the selected irreducible polynomial chosen

(b) Choosing the Constructor Polynomial of the Field

Amongst the polynomials given we have to choose between P3 and P4 as candidates, since the highest degree of the constructor should be less than 128 (i.e X^7 is the highest degree element in the polynomial possible)

$$P4(0) = 1$$

$$P4(1) = 0$$

P4 is reducible (has roots), so we cannot choose it.

$$P3(0) = 1$$

$$P3(1) = 1$$

Therefore P3 is the polynomial to choose.

(c) Implementation of the Field

Starting with both addition and subtraction, they both can be represented as a simple bitwise XOR truncated over 2^7 .

For the multiplication, we implemented the functions `mulByX` and `mul` discussed in class:

```

// multiply: mulByX iteratively
f128 mulByX(f128 x){
    f128 b = x << 1;
    if(x & 64)
        b ^= IRR_POLY;
    return b & 0x7F; // in all returned values, I will mask with 0x7f
(0b01111111) to bind it as 2^7 field and ignore the leftover overflowed
values
}

f128 mul(f128 x, f128 y){
    f128 c = 0; // accumulator
    f128 g = x;
    for(int i = 0; i < 7; i++){
        if(y & 1)
            c ^= g;
        g = mulByX(g);
        y = y >> 1;
    }
    return c & 0x7F;
}

```

We just check the MSB at $2^6 = 64$ as our polynomial of degree 7 gates that, then we xor with the irreducible polynomial we chose as a modulo.

For the inverse we adapted an implementation we found online[7] and adapted it to use bitwise operations rather than pure arithmetic operations. The reason we chose such an implementation is due to the iterative nature of the loop unlike the one we used previously which is recursive and would make bitwise calculations harder to track (it is more or less similar to the one used in ex1 for the multi-precision part).

Last we implemented a division function that makes use of the inverse function according to this property:

$$a/b = a * b^{-1}$$

So it simply translates to:

```

f128 division(f128 x, f128 y){
    return mul(x, inverse(y));
}

```

(d) f128 Demo

This demo is simple in nature and just showcases simple operations, and also confirms whether $a/a^{-1} = 1$

```
./f128-demo  
a + b = 0x17  
a * b = 0x5A  
a^-1 = 0x09  
b^-1 = 0x2A  
a / b = 0x7B
```

(e) Zech-Log representation

The main concept behind this representation is to use logarithmic representations of elements in the field to more efficiently calculate multiplications:

$$a \times b = g^i \times g^j = g^{i+j}$$

(f) Condition of Polynomials in this Field

For the construction of this field, the polynomial must be primitive. In zech-log representation we say that each element can be represented in the form $a = g^i$. And since our field's order is $p = 2^7$ meaning $(p - 1) = 127$ which is prime, this means all elements of this field are primitives of the field and can generate the field.

To prove that this field has all its elements as generators, I have written a small program called *generator* which checks which elements of the field are generators:

```
./generator  
Generators:  
[0x02,0x03,0x04,0x05,0x06,0x07,0x08,0x09,0x0A,0x0B,0x0C,0x0D,0x0E,0x0F,0x10  
,0x11,0x12,0x13,0x14,0x15,0x16,0x17,0x18,0x19,0x1A,0x1B,0x1C,0x1D,0x1E,0x1F
```

```
,0x20
,0x21,0x22,0x23,0x24,0x25,0x26,0x27,0x28,0x29,0x2A,0x2B,0x2C,0x2D,0x2E,0x2F
,0x30,0x31,0x32,0x33,0x34,0x35,0x36,0x37,0x38,0x39,0x3A,0x3B,0x3C,0x3D,0x3E
,0x3F,0x40,0x41,0
x42,0x43,0x44,0x45,0x46,0x47,0x48,0x49,0x4A,0x4B,0x4C,0x4D,0x4E,0x4F,0x50,0
x51,0x52,0x53,0x54,0x55,0x56,0x57,0x58,0x59,0x5A,0x5B,0x5C,0x5D,0x5E,0x5F,0
x60,0x61,0x62,0x6
3,0x64,0x65,0x66,0x67,0x68,0x69,0x6A,0x6B,0x6C,0x6D,0x6E,0x6F,0x70,0x71,0x7
2,0x73,0x74,0x75,0x76,0x77,0x78,0x79,0x7A,0x7B,0x7C,0x7D,0x7E,0x7F]
Total: 126
```

And sure enough, all elements are generators (0x01 is always a generator)

(g) Zech-Log Implementation

For this implementation we relied on a description of how AES makes use of logarithm and exponential tables[8]. To populate the exponential table, we first calculate all the exponents of the generator via the multiplication function we implemented earlier.

Then we can populate the logarithm table with reverse lookups using the exponentials table we just populated. We also populate a third table which we called zech-addition table that will aid us doing the additions later. Each entry is equal to $1 + g^i$

The multiplication, division and inverse functions were simple manipulations over the exponents:

$$\text{multiplication} : a * b = g^i * g^j = g^{i+j}$$

$$\text{inverse} : (g^i)^{-1} = g^{127-i}, \text{exponent wraps around like mod}$$

$$\text{division} : g^i / g^j = g^{i-j} = g^{(i-j+127) \bmod 127} \text{ ensure positive exponents}$$

$$\text{In addition: } g^i + g^j = g^i \times (1 + g^{j-i})$$

Here we can make use of the zech addition table we created to pull up $1 + g^c$

(h) Zech-Log Demo

For the demo, we created a test where we compare the previous bitwise operations to the zech-log operations and see if the results match:

```
./f128-zech-demo
Bitwise F128 operations:
a + b = 0x15
a * b = 0x44
a^-1 = 0x5C
b^-1 = 0x61
a / b = 0x65
a * a^-1 = 0x01
b * b^-1 = 0x01

Zech-log in F128 operations:
a + b = 0x15
a * b = 0x44
a^-1 = 0x5C
b^-1 = 0x61
a / b = 0x65
a * a^-1 = 0x01
b * b^-1 = 0x01

Addition matches!
Multiplication matches!
Inverses match!
Division matches!
```

Appendix

Links and Resources

[1] Karatsuba's multiplication implementation:

<https://gist.github.com/kebunit/4fa83273090837fc1157>

[2] Extended Euclidean Division Via GCD: <https://stackoverflow.com/q/59536376>

[3] Appropriate public exponent choice for RSA:

<https://crypto.stackexchange.com/a/76260>

[4] tiny-bignum-c: <https://github.com/kokke/tiny-bignum-c>

[5] tiny-bignum-c License: <https://github.com/kokke/tiny-bignum-c/blob/master/LICENSE>

[6] tiny-bignum-c author: <https://github.com/kokke>

[7] Extended euclidean algorithm: <https://stackoverflow.com/a/70501399>

[8] Calculating AES Log Table: <https://crypto.stackexchange.com/a/21174>

Compiling the project with multi-precision

The default option (all) for the Makefile is to compile without multi-precision integers. To enable them set the variable MULPR=1 and set a word size if you wish as so:

```
make MULPR=1 WORD_SIZE=4
```

WORD_SIZE here has the options (1, 2 or 4). It dictates how many bytes should each array store and how the whole integer can be chunked. The integer is over 128 bytes (1024 bits) overall.

Project structure

- |— ex1.c -> main implementation of ex1
- |— ex1.h -> header file for 64-bit functionality of the ring
- |— ex1-mulpr.h -> header for the multi-precision functionality
- |— ex2.c -> f128 bitwise implementation of the field
- |— ex2.h -> header file of f128 bitwise implementation
- |— ex2-zechlog.c -> zech-log implementation of the field
- |— ex2-zechlog.h -> header file associated with zech-log implementation
- |— f128-demo.c -> demo of f128 bitwise functionality
- |— f128-zech-demo.c -> demo of zech-log functionality
- |— generator.c -> generator checker program for f128
- |— Makefile -> Makefile of the project
- |— rsa.c -> RSA implementation of ex1
- |— rsa-demo.c -> demo of the RSA functionality
- |— rsa.h -> header file associated with RSA functions
- |— test-znz.c -> ex1 ring test demo